

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

ULAŞIMDA YAPAY ZEKÂ YARIŞMASI
KRİTİK TASARIM RAPORU



TAKIM ID: 397032

İÇİNDEKİLER

1. TAKIM ŞEMASI	4
2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ	4
3. ALGORİTMALAR VE SİSTEM MİMARİSİ	4
3.1. VERİ SETLERİ	4
3.2. ALGORİTMALAR	5
3.2.1. FASTER R-CNN	5
3.2.2. SSD	6
3.2.3. YOLO	6
4. ÖZGÜNLÜK	7
5. SONUÇLAR VE İNCELEME	7
6. REFERANSLAR	11



ŞEKİL LİSTESİ

Şekil 1: Proje Çözümü İçin Temel Akış Çizelgesi.....	5
Şekil 2: Faster R-CNN Mimarisi.....	5
Şekil 3: YOLO'nun Görüntü İzgaralama Modeli.....	6
Şekil 4: Mish ve ReLU Aktivasyon Fonksiyonlarının Grafikleri.....	6
Şekil 5: SAHI Kütüphanesi Sonuçları.	7
Şekil 6: Veri Oranları.....	7
Şekil 7: Veri Seti İçerisinden Örnek Görseller.	8
Şekil 8: Model Test Çıktıları.....	9
Şekil 9: IoU Hesaplama Mantığı	11

TABLO LİSTESİ

Tablo 1: Sınıfların Toplam Etiket Sayısı.....	4
Tablo 2: Algoritmaların Avantaj ve Dezavantaj Tablosu.....	8
Tablo 3: Model Sonuçları Tablosu.	9
Tablo 4: Gerçek Pozitif (TP) ve Gerçek Negatif (TN) Değerleri Tablosu.	10

KISALTMA LİSTESİ

Bkz: Bakınız

IoU: Intersection over Union

R-CNN: Region-Based Convolutional Neural Networks

SPP-NET: Spatial Pyramid Pooling NET

SSD: Single Shot Detector

UAİ: Uçan Ambulans İniş

UAP: Uçan Araba Park

YOLO: You Only Look Once

1. TAKIM ŞEMASI

GaiaTech yeni oluşturulmuş bir takım olmasına rağmen, alanında istekli, meraklı ve uzmanlaşmış kişilerden oluşmaktadır. Hedefimiz ulaşım, sağlık, güvenlik ve eğitim gibi alanlarda milli teknoloji hamlesi vizyonuna destek olup insanlık yararına milli yazılımlar üretmektir. Yapay Zekâ teknolojilerinin geliştirilmesinin ve üretilmesinin ülkemiz geleceği için önemini farkında bulunmaktayız. Bu sebepten ötürü amacımız Yapay Zekâ teknolojileriyle, ulaşımında karşılaşılabilecek problemlere çözüm üretmeye çalışmaktır. Ekibimiz Mekatronik Mühendisliği 1. sınıf öğrencilerinden oluşmaktadır. Aşağıda takımımızın görev dağılımı ve organizasyonu hakkında bilgi maddeler halinde verilmiştir.

- Kaptan: Görüntü işleme, yazılım mimarisi.
- Üye 1: Yazılım mimarisi, model eğitimi ve test.
- Üye 2: Literatür taraması, veri etiketleme ve değerlendirme, görüntü işleme.

2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ

Teknofest tarafından verilen örnek videonun her bir karesinin .jpg uzantılı resim dosyasına dönüşümü yapılmıştır. Bu görüntüler YOLOv4 formatına uygun bir şekilde tek tek etiketlenmiştir. Tek tip etiket formatında çalışıp ihtiyaç duyulduğunda farklı modeller için etiket format dönüşümü yapılmıştır. Drone ile kalabalık yerlerde kayıt alıp takımımızın veri seti zenginleştirilmiştir. Açık kaynak olan veri setlerini araştırılmış ve en uygun olan veri setleri kullanılmıştır. Suudi Arabistan da bulunan Prens Sultan Üniversitesi'nde yapılmış çalışmada kullanılan açık kaynak veri seti[1] ve VisDrone[2] veri setinden alınmış görüntüler takımımızın veri setine eklenmiştir. Uygun açıda olmayan görüntüler veri setinden çıkarılarak model doğruluğunu artırma amaçlanmış ve yarışma şartnamesine uygun olarak çalışmalarımıza devam edilmiştir. Oluşturulmuş veri seti tek bir çatı altında toplanmış ve veri inceleme yöntemleri ile hangi sınıftan kaç adet etiketleme olduğu incelenmiştir. Sonuçlara aşağıdaki tabloda [Tablo 1] yer verilmiştir. Sonuçlar incelendiğinde UAP ve UAİ sınıfındaki veriler yetersiz görülmüştür. Sentetik bir şekilde veri çoğaltma teknikleri ile veriler çoğaltılmış veri setine eklenmiştir. Model eğitimi sürecinde YOLO'nun 4 farklı türevi, SSD ve Faster R-CNN algoritmaları denenmiştir. Algoritmalar model inceleme yöntemleri ile incelenmiştir. Detaylar "5. SONUÇLAR VE İNCELEME" bölümünde bahsedilecektir.

3. ALGORİTMALAR VE SİSTEM MİMARİSİ

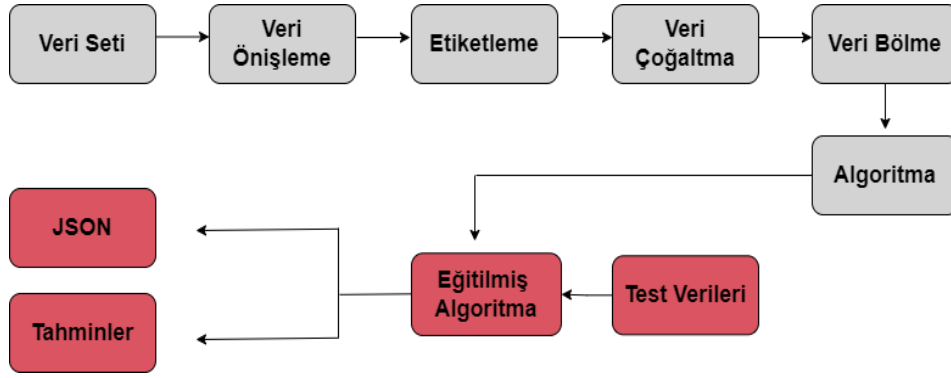
3.1. VERİ SETLERİ

	Araç	İnsan	UAP	UAİ
Etiket (Adet)	390374	168355	3236	3358

Tablo 1: Sınıfların Toplam Etiket Sayısı.

Yarışmanın amacı uçan arabalara yapay zekâ tabanlı nesne tespit sistemi ile uçan arabanın kamera verilerini kullanarak taşıt ve insan verilerini tespit etmektir. Bu nedenden dolayı kullanılacak verilerde 60-90 derece arası çekilmiş görüntüler ile veri seti oluşturulmuştur. Elimizde bulunan diğer görüntüler bu şarta göre incelenmiş ve düzenlenmiştir. Paylaşılan yarışma videosunda yer alan araç, insan, UAP ve UAİ sınıfları detaylı ve özenli bir şekilde etiketlenmiştir. Farklı hava koşullarında nesne tespiti yapabilmek ve doğruluğu arttırmak amacıyla yarışma verileri dışında veri setlerinin eklenmiştir. [1], [2] Yarışma tarafından verilenlerle birlikte toplam 51.349 adet veri bulunmaktadır. Nesnelerin tam görünmediği durumları da etiketleme yapılarak doğruluğun artırılması sağlanmıştır. Yayaların sadece ayakta olduğu verilerin yanı sıra oturur vaziyette olduğu veriler de etiketlenmiştir. UAP ve UAİ tespiti için sentetik veri üretim yöntemi ile veriler çoğaltılmıştır. Herhangi sınıf verisi olmayan görüntüler de veri setine eklenip zenginleştirilmiştir.

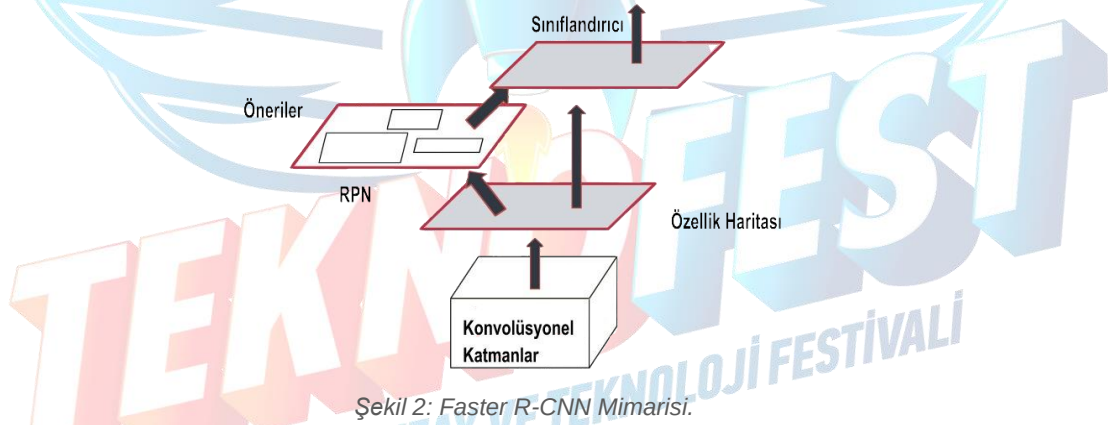
3.2. ALGORİTMALAR



Şekil 1: Proje Çözümü İçin Temel Akış Çizelgesi

Nesne tespiti açısından, iki ana tespit yöntemi bulunmaktadır. Bunlar öneri odaklı tek aşamalı ve regresyon odaklı iki aşamalı nesne tespit yöntemleridir. İki aşamalı nesne tespiti, yüksek hassasiyette tespit avantajına sahiptir ancak iki aşamalı olmasından kaynaklı olarak tek aşamalı yöntemlere göre daha yavaş kalmaktadır. [3] YOLO'yu diğer algoritmalarından ayıran önemli özelliklerden biri gerçek zamanlı nesne tespiti yapması ve tek aşamalı nesne tespit algoritması olmasıdır. Yapılan çalışmalar incelendiğinde YOLOv4 algoritmasının doğruluğu diğer algoritmalarla karşılaştırıldığında daha iyi olduğu tespit edilmiştir. [4] Projemiz için en uygun algoritma YOLOv4 olduğuna karar verilmiştir. 3. Bölümün alt başlıklarında modeller daha detaylı incelenmiştir. "5. Sonuçlar ve İnceleme" bölümünde modellerin avantaj ve dezavantajları tablolanmıştır.

3.2.1. FASTER R-CNN



Şekil 2: Faster R-CNN Mimarisi.

Faster R-CNN iki aşamadan oluşan nesne tespiti algoritmasıdır. İlk aşamada bölge teklif önerisiyle nereye bakılacağını bildiren derin ve tam konvolüsyonel katman bulunmaktadır. İkinci aşamada öneriden bölgeleri kullanan hızlı R-CNN algılayıcısı bulunmaktadır. İlk aşamadan gelen tahmin konumlarını kullanarak nesne tespiti için bölge teklif önerisinde bulunur.[5] Bölge teklif ağı (RPN) bir görüntüyü (herhangi bir boyutta) girdi olarak almakta ve her biri bir nesne puanına sahip bir dizi dikdörtgen nesne teklifi çıkarmaktadır. Bu işlemi tam bir konvolüsyon ağı ile modeller ve hesaplamayı Fast R-CNN nesne algılama ağı ile paylaşmaktadır.[6]

Şekil 2'de Faster R-CNN mimarisi gösterilmektedir. Faster R-CNN algoritması kenar kutuları gibi harici bir algoritma kullanmak yerine bölge tekliflerini doğrudan ağı aktarmak için RPN katmanı eklenmiştir. RPN nesne tespiti için bağlantı kutuları kullanmaktadır. Bu sayede R-CNN yapısının nereye bakacağını belirlemektedir. Ağda bölge teklifleri oluşturur ve verileri bunlara göre daha iyi değerlendirmektedir. [7]

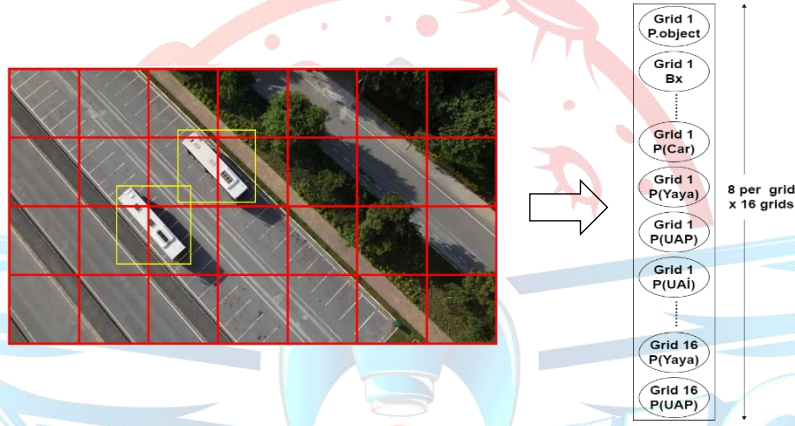
Faster R-CNN diğer modellerle kıyaslandığında doğruluk oranı olarak performans gösterse de YOLOv4 modeline göre çok yavaş kalmaktadır. Projenin amacı gerçek zamanlı nesne tespiti olduğu için Faster R-CNN modeli yarışmada kullanılması uygun bulunmamıştır.

3.2.2. SSD

SSD algoritması 2 bölümden oluşur. Bunlar özellik haritalarının çıkarılması ve nesneleri algılamak için konvolüsyonel filtrelerin uygulanmasıdır. SSD algoritmasının arkasındaki temel mantık, sabit boyutlu sınırlayıcı kutu tahmini üretmektir. Bu kutulardaki nesne sınıfı örneklerinin varlığı için puanlar verir ve en iyi tahminleri çıktı olarak veren ileri beslemeli konvolüsyonel ağ kullanılmaktadır. [8]

SSD modeli şu şekilde çalışmaktadır. Giriş görüntüsünü çeşitli boyutlarda ızgaralara bölerek farklı sınıflar ve farklı en boy oranları için tahminde bulunmaktadır. Tahminler arasından değerlendirme yapılmaktadır. Yüksek puanlı ve eşleşen tahminlere karar verilmektedir. Çalışma mantığı olarak incelendiğinde YOLO ve SSD modeli benzerdir. Resmi tek aşamalı olarak inceler ve görüntüleri tek aşamada algılamaktadır.[9] SSD modeli, bir temel ağı sonuna, farklı ölçeklerin ve en boy oranlarının varsayılan kutularının ofsetleri ve bunlarla ilişkili puanları tahmin eden birkaç özellik katmanı eklemektedir. SSD, sınıf başına ortalama 8732 tahmin üretirken; YOLO, sınıf başına yalnızca 98 tahmin üretmektedir. Bu durumlardan dolayı nesneler üzerinde SSD algoritmasının başarı oranı düşük çıkmaktadır. Çok tahminde bulunduğundan kaynaklı hız açısından olumsuz sonuçlar alınmıştır.[8] SSD modeli bu sebeplerden dolayı tercih edilmemiştir.

3.2.3. YOLO

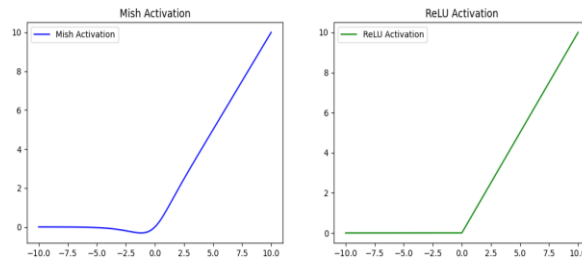


Şekil 3: YOLO'nun Görüntü Izzaralama Modeli.

Nesne tespiti modelleri iki kategoride incelenmektedir. Bu modeller tek aşamalı ve iki aşamalı modellerdir. İki aşamalı modeller nesne lokalizasyonu ve sınıflandırmasını birbirinden ayrı bir şekilde yaparken tek aşamalı modeller aynı anda tahminlerde bulunmaktadır. Bu mantık ile görseli tek aşamada değerlendirip sadece bir kez bakmaktadır. YOLOv4 de bu mantık ile çalışan tek aşamalı modellerden biridir.[10]

YOLO'nun nesne tespiti özelliği, birden çok ağı entegrasyonu sayesinde daha da geliştirilmiştir. YOLO algoritması, görüntüyü girdi olarak alarak başlamaktadır. Birbirine komşu ve aynı sınıfları gösteren bölümleri birleştirerek sınırlayıcı kutu oluşturmaktadır.[11] Görüntüyü ızgaralara bölerek her ızgara için puanlama yapmaktadır. (Bkz. Şekil 3) Sınırlayıcı kutular tahminlerin hesaplanması sonucu oluşmaktadır. Buna göre YOLO her ızgara için ayrı bir tahmin vektörü oluşturmaktadır. Son tahminler yapılmaktadır. Sınıflandırma ile farklı ölçeklere ve boyutlara sahip çoklu sınırlayıcı kutular tahmin edilmektedir. [12]

YOLOv3 ReLU aktivasyon fonksiyonu kullanmaktadır. Bu durum olumsuz sonuçlanmaktadır. YOLOv4 gelenekselleşen ReLU aktivasyon fonksiyonu yerine Mish fonksiyonu kullanmaktadır.[13] (Bkz. Şekil 4.) Bu durum modelin kesin kurallar yerine hata payı bırakarak "5. SONUÇLAR VE İNCELEME" açıkladığımız IoU değerinin artmasına sebep olmaktadır.



Şekil 4: Mish ve ReLU Aktivasyon Fonksiyonlarının Grafikleri.

YOLO'yu diğer algoritmalarından ayıran en önemli özelliği gerçek zamanlı nesne tespiti yapmasıdır. YOLO hem performans hem başarı oranının diğer algoritmalarla göre yüksek olmasından dolayı gerçek zamanlı nesne tespiti çalışmalarında tercih edilmektedir. [14] Çalışma mantığından kaynaklı diğer algoritmalarla göre çok daha verimli çalışmaktadır. Bu nedenlerden dolayı yarışmada YOLOv4 algoritmasının kullanılmasına karar verilmiştir.

4. ÖZGÜNLÜK

Açık kaynak kodlu SAHI kütüphanesini kullanarak alınan görüntüyü, parçalı biçimde tüm görüntü üzerinde detayları yakalama konusunda iyileştirme yapılmıştır. Aşağıdaki görselde yaptığımız çalışmaya örnek bir çalışma bulunmaktadır. [15]



Şekil 5: SAHI Kütüphanesi Sonuçları.

Görüntü kalitesi yüksek olan resimleri modelin yeniden boyutlandırıp detayları kaybetmesini engellemek amacıyla SAHI kütüphanesi kullanılmaktadır. İlk aşama olarak modelin istediği ölçülerde resmi parçalara ayırmak için gerekli hesaplama yapılmaktadır. Fotoğrafın kaç parçaya ayrılması gerektiğine karar verildikten sonra yeni oluşan etiketlemelerin koordinatları hesaplanır ve yeniden etiketleme yapılmaktadır. Bu sayede görüntü üzerinde veri kaybını önlemektedir.

Image Ops tekniği sayesinde görüntüyü etiketlemek için birçok yöntem kullanılabilir. Kontrast eşitlemesi Autocontrast ile yapılırken; Equalize ile de görüntünün histogramı eşitlenmektedir. Görüntünün tersini elde etmek için Invert, görüntüdeki renk kanallarının bit değerlerini düşürmek için Posterize ya da belirlenen eşik değerine göre görüntünün piksellerinin tersini almak için Solarize yöntemi kullanılabilir. [16]

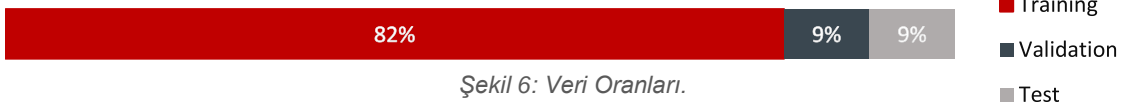
Rastgele seçilen iki görüntüden Cutout tekniğindeki gibi parçalar çıkarılmaktadır. Çıkarılan parçalar birbiriyle değiştirilir ve görüntüye yapıştırılmaktadır. Bu şekilde Cutout tekniğindeki gibi boş alanı doldurmak için renk yerine diğer görüntüden alınan parça kullanılmaktadır.

Özellikle derin öğrenme bağlamında kullanılan veri büyütme, daha fazla etiketli veri oluşturmanın basit bir yolunu sağlar; genellikle döndürme, çevirme, renk kayması veya görüntünün ve etiketin geçerliliğini koruyan diğer benzer iki boyutlu manipülasyonlar anlamına gelmektedir. Biz de özgünlük adına, verilerimizde rastgele kontrast/parlaklık değişimi, gürültü ve bulanıklık, döndürme, perspektif projeksiyonu ve kumaş desenleri uygulanmıştır. [17]

Özgünlük kısmında belirtilen çalışmalar yazılım mimarisinde denenmiş olup olumlu sonuç gösteren çalışmalar raporda belirtilmiştir. Yukardaki yöntemler eğitim sürecinde kullanılmış olup olumlu yönde etki göstermiştir.

5. SONUÇLAR VE İNCELEME

Yarışmada kullanılmak için YOLOv4 algoritması belirlenmiştir. YOLOv4 algoritması 416x416 çözünürlüğünde görseli girdi olarak kullanılmaktadır.[3] Görüntü üzerinde, özgünlük kısmında değinilen mozaik işlemi yapılarak veri kaybını en az seviyeye çekilmiştir. Model eğitiminde toplam 51.349 veri kullanılmıştır. Verilerimizi training, validation ve test olmak üzere 3 ayrı bölüme ayrılmıştır.



Şekil 6: Veri Oranları.

Verilerin eşit dağılması için kendi hazırladığımız algoritma ile verileri dağınık bir şekilde bölümler oluşturulmuştur. Oluşturulan bölümlerin oranları Şekil 6'da belirtilmiştir. Model eğitirken 6000 epoch yapılmıştır. Modelin ezberleme ihtimaline karşı early stop kullanılmıştır. [18]

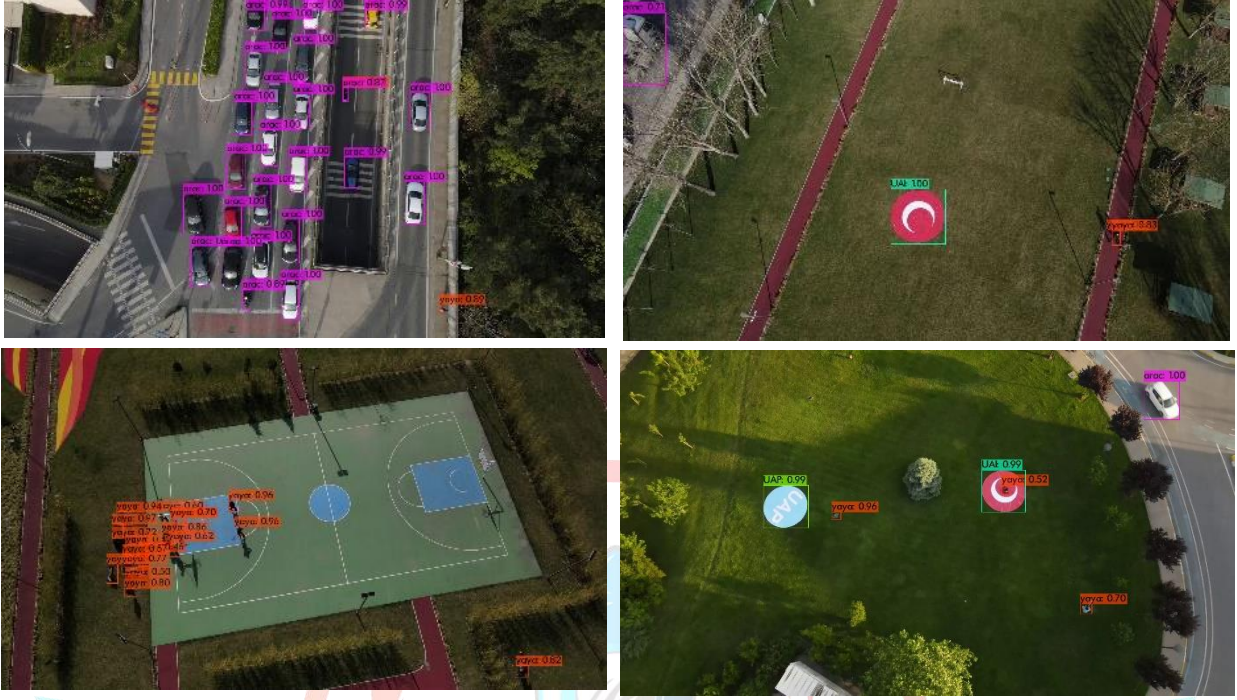
Futbol ve basketbol sahalarındaki iniş kalkış yerlerine benzer hatalı öğrenmesine sebep olacak veriler etiketsiz olarak eklenip hata oranı azaltılması amaçlanmıştır. Bunlara ek olarak trafik dubası, futbol kalesi, sokak lambası tespit edildiği de gözlemlenmiştir veri kümesini zenginleştirerek bu sorun aşılmıştır. (Şekil 7.)



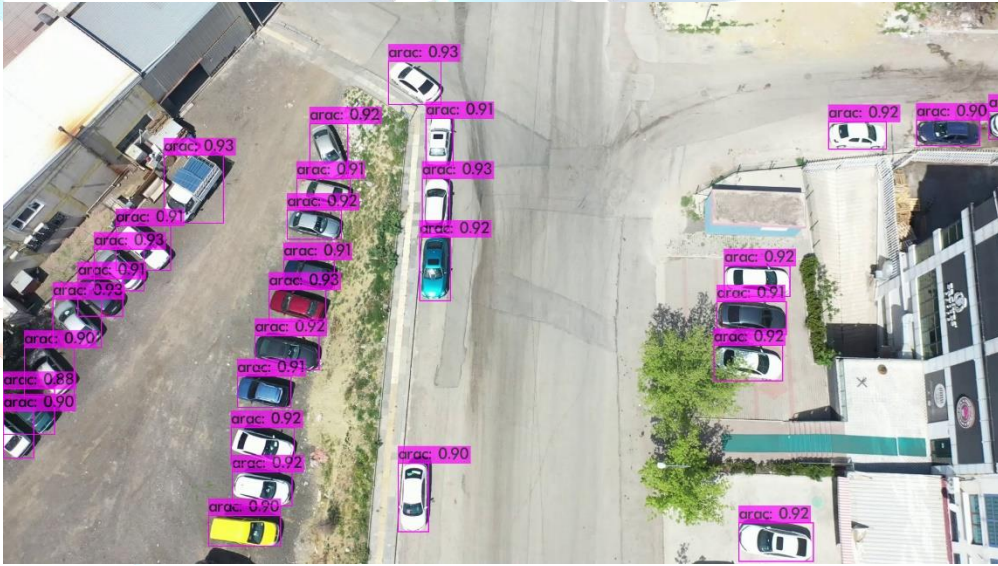
Şekil 7: Veri Seti İçerisinden Örnek Görseller.

Tablo 2: Algoritmaların Avantaj ve Dezavantaj Tablosu.

Model	Avantajları	Dezavantajları
R-CNN	Ağ, nesne algılama problemini sınıflandırma problemine dönüştürür ve doğruluğu büyük ölçüde iyileştirir.	Her tespit hedefinden kısmen örtüşen aday alanlar üretir. YOLOv4'e kıyasla çok yavaştır
SPP-Net	Son evrişim katmanından sonra uzamsal piramit havuzlama katmanını tanıtır, böylece tekrarlayan işleme ortadan kalkar.	Uzun eğitim süresi olan çok aşamalı bir süreçtir.
Fast R-CNN	Yağmur yağması ve test edilmesi SPP-Net'ten önemli ölçüde daha hızlıdır.	Ağ hala aday bölge seçim algoritmasına bağlıdır. YOLOv4'e kıyasla çok yavaştır.
SSD	Çok ölçekli özellik haritası benimsenmiştir ve işlem hızı hızlıdır.	Küçük nesne algılama konusunda zayıftır.
YOLO	Ağ, tam görüntüyü Bağlam bilgisi olarak kullanarak gerçek zamanlı gereksinimleri karşılayabilir.	Nesnenin ölçeğine nispeten duyarlıdır. Küçük hedef tespitinin etkisi iyi değildir. Bu durum veri kaybını azaltarak ortadan kaldırılabilir.



Şekil 8: Model Test Çıktıları.



Tablo 3: Model Sonuçları Tablosu.

Model	Doğruluk	Doğrulama Doğruluğu	Kayıp	Doğrulama Kaybı
YOLOv4	0.8321	0.881	0.7715	0.7256
YOLOv3	0.7124	0.832	0.6985	0.7364
Faster R-CNN	0.7426	0.7424	0.7551	0.8645
SSD	0.6952	0.7541	0.6722	0.7045

Tablo 4: Gerçek Pozitif (TP) ve Gerçek Negatif (TN) Değerleri Tablosu.

SINIFLAR	TP	TN
Araç	65353	22452
Yaya	15620	13054
UAP	657	26
UAI	794	38

Değerlendirme kriterleri olarak, karışıklık matrisi (confusion matrix), duyarlılık (recall), hassasiyet (precision), f1 puanı (f1-score) ve kesişim bölgeleri (IoU) kullanılacaktır. [19]

Karışıklık Matrisi: Karışıklık matrisi, bir sınıflandırma algoritmasının performansını tanımlamak için kullanılan tablodur. Karışıklık matrisi, bir sınıflandırma algoritmasının performansını görselleştirir ve özetlemektedir.

Karışıklık matrisi, sınıflandırıcının ölçüm metriklerini tanımlamak için kullanılan dört temel özellikten oluşmaktadır. Bu özellikler şunlardır:

TP (True Positive/ Gerçek Pozitif): Gerçek değerleri 1 olarak sınıflandırılmaktadır. Gerçek pozitif, 1 olarak sınıflandırılan ve gerçekten 1 olan değerlerin sayısıdır.

TN (True Negative/ Gerçek Negatif): Gerçek olmayan değerleri 0 olarak sınıflandırılmaktadır. 0 olarak sınıflandırılan ve gerçekten de 0 olan değerlerin sayısıdır.

FP (False Positive/ Yanlış Pozitif): 1 olarak sınıflandırılan fakat gerçekte 0 olan değerlerin sayısıdır.

FN (False Negative/ Yanlış Negatif): 0 olarak sınıflandırılan fakat gerçekte 1 olan değerlerin sayısıdır.

Bir algoritmanın performans ölçütleri, yukarıda belirtilen TP, TN, FP ve FN temelinde hesaplanan doğruluk, hassasiyet, duyarlılık ve F1 puanıdır.

Doğruluk, sınıfların ortalama hassasiyet (AP) değerlerinin aritmetik ortalaması (mAP) alınarak bulunmaktadır.[20] Denklem 1'de formülleri gösterilmektedir.

$$mAP = \sum_{c \in classes} \frac{\#TP(c)}{\#TP(c) + \#FP(c)} \quad (1)$$

$$AP = \left(\frac{1}{n}\right) \sum_{Recall_i} Precision(Recall_i) = 1$$

Duyarlılık, isabet oranı olarak bilinmektedir ve yüksek olması beklenmektedir. Sınıflayıcının ne kadar TP ya da TN değeri doğru tahmin ettiğinin bir ölçüsüdür. Denklem 2'de duyarlılığın nasıl hesaplandığı gösterilmiştir.

$$Duyarlılık = \frac{TP}{TP + FN} \quad (2)$$

Hassasiyet, pozitif olarak sınıflandırılanlar arasında doğru sınıflandırılmış örneklerin oranını değerlendirir. Duyarlılık gibi hassasiyetin de yüksek olması beklenmektedir. Denklem 3'de hassasiyetin nasıl hesaplandığı gösterilmiştir.

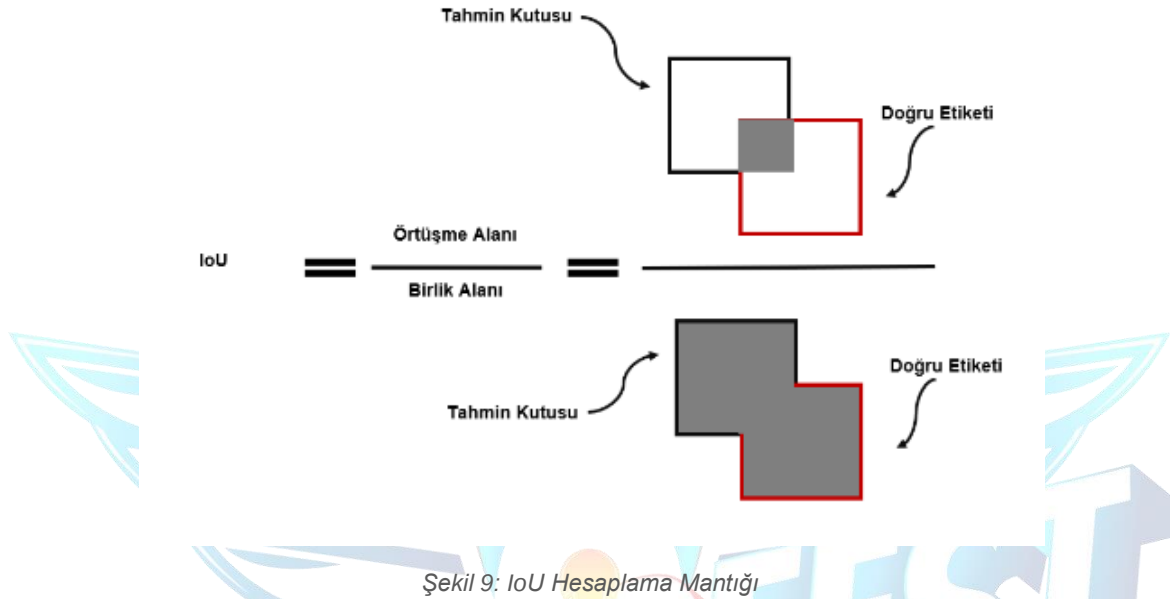
$$Hassasiyet = \frac{TP}{TP + FP} \quad (3)$$

F-1 puanı, sınıflandırıcının ne kadar iyi performans gösterdiğinin bir ölçüsüdür. Duyarlılık ve hassasiyetin harmonik ortalamasıdır. Denklem 4’de f-1 puanının nasıl hesaplandığı gösterilmiştir.

$$F1 = 2 \times \frac{\text{Hassasiyet} \times \text{Duyarlılık}}{\text{Hassasiyet} + \text{Duyarlılık}}$$

(4)

IoU, her bir sınırlayıcı kutu için, tahmin edilen sınırlayıcı kutu ile gerçek sınırlayıcı kutu arasındaki örtüşmeyi ölçmek için kullanılır. Şekil 9’de yapı gösterilmektedir.



Modelin sonuçları ve tablolar incelediğinde YOLOv4 modelinin doğruluğu başarılı olduğu düşünülmektedir. Bu aşamadan sonra model değişikliği yapılmayacaktır. Yarışma zamanına kadar yeni veriler bulunarak veri havuzu büyütülecek ve model geliştirilecektir.

6. REFERANSLAR

- [1] “aniskoubaa/psu-car-dataset.” <https://github.com/aniskoubaa/psu-car-dataset> (accessed Jun. 15, 2022).
- [2] “VisDrone/VisDrone-Dataset: The dataset for drone based detection and tracking is released, including both image/video, and annotations.” <https://github.com/VisDrone/VisDrone-Dataset> (accessed Jun. 15, 2022).
- [3] A. Ammar, A. Koubaa, M. Ahmed, A. Saad, and B. Benjdira, “Vehicle detection from aerial images using deep learning: A comparative study,” *Electronics (Switzerland)*, vol. 10, no. 7, pp. 1–31, 2021, doi: 10.3390/electronics10070820.
- [4] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, “A Review of Yolo Algorithm Developments,” *Procedia Computer Science*, vol. 199, pp. 1066–1073, 2021, doi: 10.1016/j.procs.2022.01.135.
- [5] P.- Sedán *et al.*, “DERİN ÖĞRENME ALGORİTMALARI KULLANARAK İNSANSIZ HAVA ARAÇLARI İLE SİLAH TESPİTİ,” *Journal of Chemical Information and Modeling*, vol. 21, no. 1, pp. 1–9, 2020, [Online]. Available: <https://doi.org/10.1016/j.tmaid.2020.101607> <https://doi.org/10.1016/j.ijisu.2020.02.034> <https://onlinelibrary.wiley.com/doi/abs/10.1111/cjag.12228> <https://doi.org/10.1016/j.ssci.2020.104773> <https://doi.org/10.1016/j.jinf.2020.04.011>

- [6] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017, doi: 10.1109/TPAMI.2016.2577031.
- [7] L. Zhang, L. Lin, X. Liang, and K. He, "Is faster R-CNN doing well for pedestrian detection?," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 9906 LNCS, pp. 443–457, 2016, doi: 10.1007/978-3-319-46475-6_28/TABLES/5.
- [8] W. Liu *et al.*, "SSD: Single shot multibox detector," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 9905 LNCS, pp. 21–37, 2016, doi: 10.1007/978-3-319-46448-0_2/FIGURES/5.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 8691 LNCS, no. PART 3, pp. 346–361, 2014, doi: 10.1007/978-3-319-10578-9_23.
- [10] K. Wang and M. Liu, "YOLO-Anti: YOLO-based counterattack model for unseen congested object detection," *Pattern Recognition*, p. 108814, 2022, doi: 10.1016/j.patcog.2022.108814.
- [11] S. Shinde, A. Kothari, and V. Gupta, "YOLO based Human Action Recognition and Localization," *Procedia Computer Science*, vol. 133, no. 2018, pp. 831–838, 2018, doi: 10.1016/j.procs.2018.07.112.
- [12] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," 2020, [Online]. Available: <http://arxiv.org/abs/2004.10934>
- [13] D. Misra, "Mish: A Self Regularized Non-Monotonic Activation Function," 2019, [Online]. Available: <http://arxiv.org/abs/1908.08681>
- [14] G. Plastiras, C. Kyrkou, and T. Theodoridis, "Efficient convnet-based object detection for unmanned aerial vehicles by selective tile processing," *ACM International Conference Proceeding Series*, 2018, doi: 10.1145/3243394.3243692.
- [15] "obss/sahi: A lightweight vision library for performing large scale object detection/ instance segmentation." <https://github.com/obss/sahi> (accessed Jun. 15, 2022).
- [16] H. Li, W. Luo, X. Qiu, and J. Huang, "Identification of Various Image Operations Using Residual-Based Features," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 28, no. 1, pp. 31–45, Jan. 2018, doi: 10.1109/TCSVT.2016.2599849.
- [17] B. Zoph, E. D. Cubuk, G. Ghiasi, T. Y. Lin, J. Shlens, and Q. v. Le, "Learning Data Augmentation Strategies for Object Detection," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 12372 LNCS, pp. 566–583, 2020, doi: 10.1007/978-3-030-58583-9_34.
- [18] L. Prechelt, "Early stopping - But when?," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 7700 LECTURE NO, pp. 53–67, 2012, doi: 10.1007/978-3-642-35289-8_5/COVER/.
- [19] J. L. Borade and M. A. Lakshmi, "Multi-class object detection system using hybrid convolutional neural network architecture," *Multimedia Tools and Applications*, pp. 1–25, Apr. 2022, doi: 10.1007/S11042-022-13007-7/TABLES/9.
- [20] A. M. Roy, R. Bose, and J. Bhaduri, "A fast accurate fine-grain object detection model based on YOLOv4 deep neural network," *Neural Computing and Applications*, vol. 34, no. 5, pp. 3895–3921, 2022, doi: 10.1007/s00521-021-06651-x.